Claude Certified Architect Foundations

Score Report

CANDIDATE: Yogendhar Bolisetti

CANDIDATE ID: ANTH516426

REGISTRATION NUMBER: 548192953

DATE: Saturday, October 3, 2026

SITE NUMBER: Proctored

EXAM CODE: CCAR-F

EXAM: Claude Certified Architect Foundations

This report shows the percentage of items in each section you answered correctly for the Claude Certified Architect Foundations

PASSING SCORE: 720 **YOUR SCORE: 960** **GRADE: Pass**

TEST OBJECTIVES	PERCENT CORRECT
Design orchestration-layer safeguards ensuring every agent session ends with a completed resolution or human escalation, regardless of how the agentic loop terminates.	100%
Design structured handoff packages that preserve accumulated context, findings, and authorization state when transferring control between agent steps or to a human operator.	100%
Apply session resumption techniques—including targeted re-analysis of changed files and context injection—to restore agent state accurately without repeating prior work.	100%
Decompose complex tasks into dynamically generated subtasks that adapt as new information is discovered, rather than executing a fixed sequence regardless of intermediate findings.	100%
Apply PreToolUse and PostToolUse hook patterns to enforce business rules and policy constraints not delegated to model discretion.	100%
Explain how the agentic loop uses model responses and stop_reason signals to decide whether to continue tool execution or terminate and return a final response.	100%
Create and deploy custom slash commands in the correct project or user directory so they are available to intended users and invoked on demand for task-specific workflows.	100%
Structure iterative refinement workflows by providing concrete input-output examples, targeted feedback on specific failures, and batched issue descriptions for consolidated evaluation.	75%
Determine when to use plan mode versus direct execution based on task scope, reversibility, architectural uncertainty, and the need for stakeholder review before implementation.	100%
Distinguish instructions that must be enforced through settings permissions or hooks from those appropriately placed in CLAUDE.md, and restructure configurations accordingly.	100%
Select the correct Claude Code configuration mechanism—CLAUDE.md, .claude/rules/ with glob patterns, Skills, hooks, or settings permissions —based on guidance type and when it should apply.	100%

Implement PostToolUse hooks that automatically enforce code quality constraints—such as formatting, linting, or test execution—after every file edit, independent of model instruction-following.	100%
Apply systematic codebase exploration strategies using Grep, Glob, and Read tools that build incremental understanding while managing context window constraints.	100%
Configure MCP servers and project settings at the correct scope—project-level for shared team tooling and user-level for personal or experimental configurations.	100%
Apply context management strategies—subagent isolation, scratchpad files, and targeted file reading—to sustain coherent codebase exploration across sessions exceeding context limits.	100%
Select the appropriate method for providing project context to Claude Code—@ references, CLAUDE.md, or inline description—based on reusability, specificity, and cross-session need.	100%
Explain why conversation history must be explicitly included in each API request and identify the correct mechanism for maintaining state across multiple turns in a stateless API.	50%
Apply escalation decision criteria to determine when an agent should immediately honor a human escalation request versus attempt autonomous resolution using available tools.	100%
Design human review routing strategies that direct extractions to reviewers based on confidence scores, document characteristics, and field-level ambiguity rather than random sampling.	100%
Select the appropriate API processing mode—synchronous Messages API or asynchronous Message Batches API—based on latency requirements, workflow blocking behavior, and acceptable processing windows.	100%
Select and implement the most reliable structured output method—tool use with JSON schema, prompt-based formatting, or prefilled responses —based on required schema compliance strictness.	100%
Design extraction schemas with optional fields, nullable values, and appropriate enum definitions that allow the model to accurately represent missing or ambiguous information without fabricating values.	100%
Implement tool use with defined JSON schemas to enforce structured output compliance, and configure tool_choice to guarantee tool invocation when conversational responses would cause downstream failures.	100%
Apply extraction accuracy patterns—structured schemas with optional fields, format normalization instructions, and few-shot examples—to reduce hallucination and improve consistency across varied document formats.	100%
Design feedback loop mechanisms that capture structured metadata about model errors and use those patterns to improve prompts, schemas, or few-shot examples in future iterations.	100%
Implement MCP tool error handling that surfaces structured, type-specific error information to the agent—including recoverability status and suggested next actions—rather than generic failure messages.	100%
Improve tool selection reliability by expanding tool descriptions with use-case examples, input format specifications, and explicit disambiguation guidance for semantically similar tools.	100%
Integrate MCP servers into Claude Code and agent applications by selecting the correct server scope, configuring authentication via environment variable expansion, and verifying tool discovery.	100%
Configure the tool_choice parameter to guarantee tool invocation when structured output is required, and sequence multi-tool workflows so prerequisite data is obtained before dependent tools are called.	100%